



UCSC

Herramientas de Análisis de Código y Detección de Vulnerabilidades

Informe de Trabajo Individual

Asignatura : Desarrollo avanzado

Profesor: Cesar Varela

Estudiante: Paloma chavez

Aplicacion: Gestion de Empleados (Flask + SQLite)

<https://github.com/pppaloo/DesarrolloAvanzado2>

Contenido

- 1. Introducción
- 2. Herramientas investigadas (cuadro comparativo)
- 3. Descripción del programa desarrollado
- 4. Metodología aplicada
- 5. Resultados del análisis
- 6. Análisis de hallazgos y propuestas de corrección
- 7. Conclusiones y aprendizajes
- 8. Referencias

1. Introducción

La calidad y la seguridad de un programa no se garantizan únicamente porque la aplicación "funcione". Un programa puede ejecutarse sin errores y aun así contener vulnerabilidades de seguridad, errores de programación latentes, malas prácticas y problemas de mantenibilidad que solo aparecen cuando se someten a la prueba de condiciones específicas, de usuarios malintencionados o de mantenimiento a largo plazo. Por eso, en la ingeniería de software moderna es indispensable contar con herramientas automatizadas que analicen el código fuente desde etapas tempranas del desarrollo.

En este trabajo se desarrolló una aplicación web completa (gestión de empleados) con login, registro de usuarios y CRUD sobre una base de datos SQLite, y posteriormente se aplicaron tres herramientas de análisis de código sobre el programa: Bandit (seguridad / SAST), Flake8 (estilo y errores) y PyLint (calidad general). El objetivo fue identificar vulnerabilidades, errores y malas prácticas reales, documentarlas con evidencia y proponer correcciones concretas.

Se eligió deliberadamente una implementación con errores comunes de programación principiante —concatenación de SQL, contraseñas cifradas con MD5, clave secreta hardcodeda, depuración activada— para que las herramientas de análisis tuvieran hallazgos reales y relevantes que demostraran su utilidad práctica.

2. Herramientas investigadas

2.1 Bandit

Bandit es una herramienta de análisis estático de seguridad (SAST) para Python. Escanea el código fuente y busca patrones de vulnerabilidades conocidas: inyección SQL por concatenación de cadenas, uso de hashes inseguros (MD5, SHA1), datos hardcodedos (claves, contraseñas), uso de funciones peligrosas (eval, pickle, exec), parámetros peligrosos de framework (debug=True en Flask) y la exposición del servicio en 0.0.0.0. Clasifica cada hallazgo por severidad y confianza, y los asocia con la referencia CWE correspondiente.

Ejecución: `bandit -r gestion_empleados -f json/html/txt`. Genera reportes en JSON, HTML, CSV, XML y texto plano, ideales para integrarlos en integración continua (CI).

2.2 Flake8

Flake8 es un linter que combina tres utilidades en una sola herramienta: PyFlakes (detección de errores lógicos como imports sin usar, variables no definidas), pycodestyle (cumplimiento de la guía de estilo PEP 8: espaciado, indentación, líneas en blanco) y McCabe (complejidad cicloestática de funciones). Es rápido, ligero y muy usado en el ecosistema Python.

Ejecución: `flake8 app.py --max-line-length=150`. Se enfoca en estilo y errores triviales, pero no detecta vulnerabilidades de seguridad.

2.3 PyLint

PyLint es el analizador de calidad de código más completo y configurable de Python. Va más allá del estilo: detecta errores de programación, malos olores de código (code smells), código muerto, importaciones incorrectas, nombres que chocan con builtins, excepciones demasiado generales, falta de documentación y malas prácticas. Produce una puntuación global de 0 a 10, lo que permite medir de forma objetiva la evolución del código entre versiones.

Ejecución: `pylint app.py`. Entrega una calificación numérica y un listado detallado de mensajes clasificados por severidad.

2.4 Cuadro comparativo

Característica	Bandit	Flake8	PyLint
Categoría	SAST / Seguridad	Linter / estilo	Calidad general
Qué detecta	Vulnerabilidades y fallos de seguridad	Violaciones de PEP 8 e imports mal usados	Errores, malas prácticas, código muerto
Severidad de resultados	Baja / Media / Alta	No aplica (estilo)	Mensajes C/R/W/E/F
Formato de salida	JSON, HTML, TXT, CSV, XML	Texto plano	Texto plano / color
Velocidad	Media	Muy rápida	Lenta en archivos grandes
Falsos positivos	Algunos	Muy bajos	Puede ser exigente
Ventajas	Detecta XSS, SQLi, hashes débiles, debug activo	Sencillo y rápido de integrar	Puntuación objetiva 0-10, muy configurable
Desventajas	No detecta errores de lógica	Solo estilo, no seguridad	Configuración inicial compleja

3. Descripción del programa desarrollado

La aplicación "Gestión de Empleados" es un sistema web desarrollado en Python con el framework Flask y base de datos SQLite. Cumple los requisitos mínimos obligatorios:

- **CRUD completo (Crear, Leer, Actualizar, Eliminar) sobre la entidad Empleado.** Incluye formularios para nuevo empleado, listado, edición y eliminación, así como búsqueda por nombre, apellido o cargo, y estadísticas por departamento/cargo.
- **Base de datos funcional conectada a la aplicación.** Se usa SQLite con dos tablas: usuarios (username, password, nombre_completo) y empleados (nombre, apellido, cargo, salario, departamento, fecha_ingreso, teléfono, email, usuario_id). Las tablas se crean automáticamente al iniciar.
- **Login con autenticación de usuarios (registro / inicio de sesión, manejo de contraseñas).** Permite registrar usuarios, iniciar/cerrar sesión, y restringe el acceso a las rutas de

empleados mediante sesiones. De forma intencional, la contraseña se cifra con MD5 (débil) para que las herramientas la detecten.

Estructura del proyecto:

```
gestion_empleados/
├── app.py                # Código principal (rutas, BD, lógica)
├── empleados.db          # Base de datos SQLite
├── requirements.txt       # Dependencias (Flask)
├── templates/            # Plantillas HTML (Jinja2)
│   ├── base.html, login.html, registro.html
│   ├── lista_empleados.html, form_empleado.html
│   └── estadisticas.html, perfil.html
├── static/css/estilos.css # Estilos propios
└── reportes/             # Reportes generados por las herramientas
```

4. Metodología aplicada

- 1) Investigación de herramientas.** Se estudiaron varias alternativas y se seleccionaron Bandit, Flake8 y PyLint por ser gratuitas, de código abierto y representativas de tres categorías de análisis: seguridad, estilo y calidad.
- 2) Desarrollo de la aplicación.** Se construyó el programa Flask con login, registro de usuarios y CRUD de empleados sobre SQLite, incluyendo errores intencionales y no intencionales comunes.
- 3) Aplicación de Bandit.** Se ejecutó `bandit -r gestion_empleados` sobre todo el proyecto y se exportaron reportes en formato JSON, HTML y TXT (carpeta reportes/).
- 4) Aplicación de Flake8.** Se ejecutó `flake8 app.py --max-line-length=150` y se guardó el reporte de estilo.
- 5) Aplicación de PyLint.** Se ejecutó `pylint app.py` y se registró la puntuación y la lista de mensajes.
- 6) Análisis de resultados.** Se clasificaron y priorizaron los hallazgos, se localizaron en el código y se formularon propuestas de corrección (sección 6).

5. Resultados del análisis

5.1 Bandit (seguridad)

Bandit se ejecutó sobre todo el directorio y detectó un total de **6 problemas**: 1 de severidad Alta, 3 de Media y 2 de Baja según la clasificación de la herramienta. Los hallazgos más relevantes se describen a continuación.

Hallazgo	Tipo / Prioridad	Propuesta de corrección
B324 - Uso de MD5 para cifrar contraseñas (hashlib). (app.py:49 (línea 49))	Vulnerabilidad / Severidad ALTA La función hash_password usa hashlib.md5(). MD5 es criptográficamente inseguro: puede romperse por fuerza bruta y ataques de diccionario en segundos. Almacenar contraseñas con MD5 expone las cuentas de todos los usuarios.	Usar pbkdf2_hmac de hashlib (propio de la biblioteca estándar), o mejor werkzeug.security generate_password_hash / check_password_hash con método bcrypt o scrypt.
B608 - Posible inyección SQL en el login. (app.py:54 (variable query))	Vulnerabilidad / Severidad MEDIA La consulta de autenticación se construye concatenando el nombre de usuario y la contraseña dentro de un string SQL. Un atacante puede inyectar SQL en el campo de usuario (por ejemplo: ' OR '1'='1') y acceder sin credenciales válidas.	Usar consultas parametrizadas con placeholders (?) y pasar los valores como tupla a cursor.execute().
B608 - Posible inyección SQL en la búsqueda de empleados. (app.py:243-245 (buscar_empleados))	Vulnerabilidad / Severidad MEDIA La búsqueda de empleados por término concatena el término digitado por el usuario directamente en la instrucción SQL, sin escapar caracteres. Permite inyección SQL.	Usar consultas parametrizadas y el operador LIKE con parámetros: cursor.execute("SELECT * FROM empleados WHERE nombre LIKE ?", ('%'+termino+'%',)).
B105 / B201 - Clave secreta y depuración activa. (app.py:7 y app.py:296)	Vulnerabilidad / Severidad MEDIA y ALTA La app.secret_key está	Mover la secret_key a una variable de entorno, ejecutar con debug=False en producción y escuchar

	hardcodeada en el código y la aplicación se ejecuta con debug=True y host 0.0.0.0. Esto expone el debugger de Werkzeug (ejecución de código remota) y permite falsificar sesiones.	únicamente en 127.0.0.1 cuando no se requiera exposición externa.
--	--	---

5.2 Flake8 (estilo y errores)

Flake8 detectó 140 violaciones de estilo sobre app.py. Los códigos de error más frecuentes fueron:

- **E302 / E305 - Número de líneas en blanco entre funciones.** Muy frecuente.
- **W293 / W291 - Espacios en líneas en blanco y espacios finales.** Muy frecuente.
- **E722 - Uso de except sin especificar excepción.** Apreciable: captura toda excepción.
- **E128 - Indentación de líneas de continuación.** Apreciable.
- **F401 - Import no utilizado (import os).** Bajo impacto, pero a corregir.

Importante: aunque gran parte de los avisos de Flake8 son solo de estilo, el uso de except genérico (E722) y el import no usado (F401) sí representan hábitos que conviene corregir. Evidencia completa en reportes/flake8_report.txt.

5.3 PyLint (calidad)

PyLint calificó el código con una puntuación de **6.29/10**. Los mensajes se concentran en: falta de docstrings en funciones (C0116), uso de excepciones sin especificar (W0702 / E722), redefinición de la función builtin "id" (W0622), importaciones fuera de orden (C0411) y espacios adicionales (C0303/W293).

Evidencia completa en reportes/pylint_report.txt.

6. Análisis de hallazgos y propuestas de corrección

1. **Inyección SQL en login y búsqueda (ALTA prioridad).** Vulnerabilidad crítica: los datos del usuario se concatenan en consultas SQL. Mitigación: usar siempre consultas parametrizadas con placeholders '?'. Ejemplo de corrección en el login: `cursor.execute("SELECT * FROM usuarios WHERE username=? AND password=?", (username, hashed))`.
2. **Contraseñas con MD5 (ALTA prioridad).** Almacenar contraseñas con MD5 es inseguro. Corrección: usar `werkzeug.security.generate_password_hash(password)` y `check_password_hash(almacenado, ingresada)`, que emplea PBKDF2 por defecto.
3. **Clave secreta hardcodeada (MEDIA prioridad).** La `secret_key` en el código permite falsificar sesiones. Corrección: leer de `os.environ.get('SECRET_KEY')` y nunca subirla al repositorio (usar `.env` o variables de entorno).

4. **debug=True y host 0.0.0.0 (ALTA prioridad).** Expone el depurador de Werkzeug en todas las interfaces. Corrección: app.run no debe usar debug en producción; limitar host a 127.0.0.1 y gestionar el modo mediante variables de entorno.
5. **Excepciones genéricas (MEDIA prioridad).** except sin tipo oculta errores y dificulta la depuración. Corrección: capturar excepciones específicas (p. ej. sqlite3.Error, ValueError) y registrar el error.
6. **Falta de documentación y estilo (BAJA prioridad).** Múltiples avisos de docstrings, espacios y blank lines reducen la mantenibilidad y la calidad percibida (PyLint 6.29/10). Corrección: añadir docstrings, eliminar imports no usados y aplicar flake8 hasta obtener 0 errores.

7. Conclusiones y aprendizajes

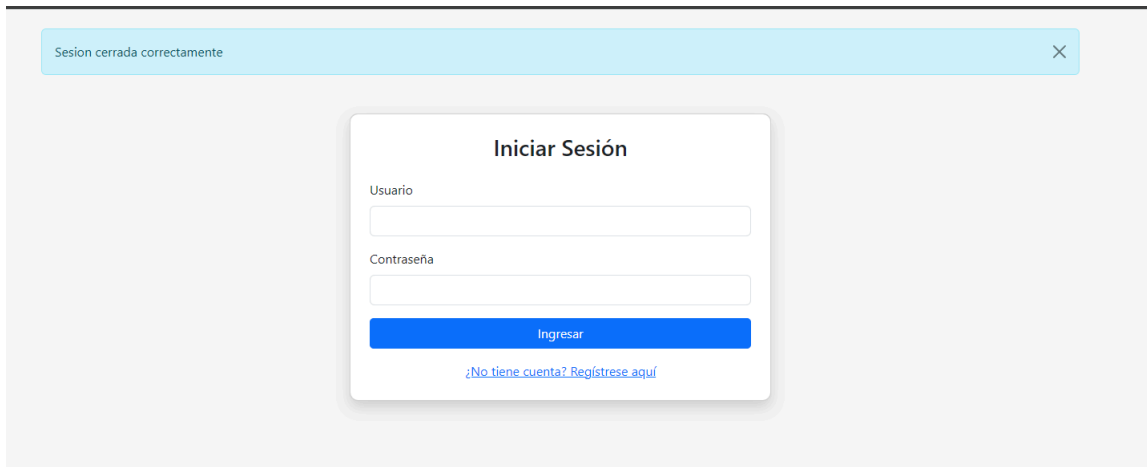
- Las herramientas de análisis de código son indispensables: detectan vulnerabilidades que los desarrolladores no vemos a simple vista, como la inyección SQL por concatenación. En este trabajo, Bandit confirmó que la aplicación, aunque funcional, tenía 6 problemas de seguridad reales.
- Cada categoría de herramienta complementa a las demás: Bandit (seguridad), Flake8 (estilo) y PyLint (calidad) aportan visiones distintas del mismo código; usarlas en conjunto da una imagen mucho más completa que cualquiera por separado.
- La priorización es clave: no todos los hallazgos tienen el mismo peso. La inyección SQL y las contraseñas con MD5 merecen corrección inmediata (alta prioridad), mientras que los avisos de estilo pueden resolverse en tandas con herramientas automáticas.
- Aprendizaje principal: integrar el análisis estático desde el inicio del ciclo de desarrollo (y en integración continua) reduce costos y evita que los problemas lleguen a producción.

8. Referencias

- Bandit (PyCQA) - Documentación oficial: <https://bandit.readthedocs.io/>
- Flake8 (PyCQA) - Documentación oficial: <https://flake8.pycqa.org/>
- PyLint - Documentación oficial: <https://pylint.readthedocs.io/>
- OWASP - SQL Injection Prevention Cheat Sheet: <https://cheatsheetseries.owasp.org/>
- OWASP - Secure coding practices: <https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/>
- PEP 8 - Guía de estilo para código Python: <https://peps.python.org/pep-0008/>
- CWE - Common Weakness Enumeration: <https://cwe.mitre.org/>

9. Anexo: Evidencia visual (capturas de pantalla)

A continuación se incorporan las capturas de pantalla que evidencian la ejecución del programa y de las herramientas de análisis. Cada figura hace referencia a la herramienta y a los reportes correspondientes (sección 5).



Esta captura muestra el login y para registrarse

Gestión Empleados

Lista

Nuevo

Estadísticas

Buscar empleado...

Buscar

paloma chavez

Salir

Empleados

+ Nuevo Empleado

ID	Nombre	Apellido	Cargo	Salario	Departamento	Fecha Ingreso	Teléfono	Email	Acciones
1	Ana	García	Gerente de Proyecto	\$45000.00	Tecnología	2021-03-15	555-1001	ana.garcia@empresa.com	Editar Eliminar
2	Luis	Martínez	Desarrollador Backend	\$32000.00	Tecnología	2022-06-01	555-1002	luis.martinez@empresa.com	Editar Eliminar
3	Maria	Lopez	Disenadora UX	\$28000.00	Tecnología	2023-01-20	555-1003	maria.lopez@empresa.com	Editar Eliminar
4	Carlos	Ramírez	Contador	\$30000.00	Contabilidad	2019-11-10	555-1004	carlos.ramirez@empresa.com	Editar Eliminar
5	Sofia	Torres	Ejecutiva de Ventas	\$24000.00	Ventas	2022-09-05	555-1005	sofia.torres@empresa.com	Editar Eliminar
6	Diego	Fernandez	Analista de Marketing	\$27000.00	Marketing	2023-05-12	555-1006	diego.fernandez@empresa.com	Editar Eliminar
7	Valentina	Castro	RRHH Senior	\$33000.00	Recursos Humanos	2020-02-25	555-1007	valentina.castro@empresa.com	Editar Eliminar
8	Jorge	Mendoza	Desarrollador Frontend	\$31000.00	Tecnología	2023-08-15	555-1008	jorge.mendoza@empresa.com	Editar Eliminar
9	Paula	Ríos	Asistente Contable	\$22000.00	Contabilidad	2024-01-08	555-1009	paula.rios@empresa.com	Editar Eliminar
10	Andres	Silva	Community Manager	\$25000.00	Marketing	2022-04-18	555-1010	andres.silva@empresa.com	Editar Eliminar

Esta captura muestra la lista de los empleados se ve su ID su nombre, apellido, cargo, salario, departamento, fecha ingreso, telefono , email y las acciones que serian editar o eliminar,

Nuevo Empleado

Nombre	Apellido
Cargo	Salario
Departamento	Fecha de Ingreso
Seleccione...	dd-mm-aaaa
Teléfono	Email

GuardarCancelar

Esta captura muestra el formulario para agregar un nuevo empleado al sistema.

Editar Empleado

Nombre	Apellido
Ana	Garcia
Cargo	Salario
Gerente de Proyecto	45000,0
Departamento	Fecha de Ingreso
Seleccione...	15-03-2021
Teléfono	Email
555-1001	ana.garcia@empresa.com

GuardarCancelar

Esta captura muestra un formulario para editar un empleado que ya este en el sistema para luego guardar los cambios.

Estadísticas de Empleados

Total Empleados
10

Salario Promedio
\$29700.00

Por Departamento

Departamento	Cantidad
Contabilidad	2
Marketing	2
Recursos Humanos	1
Tecnología	4
Ventas	1

Por Cargo

Cargo	Cantidad
Analista de Marketing	1
Asistente Contable	1
Community Manager	1
Contador	1
Desarrollador Backend	1
Desarrollador Frontend	1
Disenadora UX	1
Ejecutiva de Ventas	1
Gerente de Proyecto	1
RRHH Senior	1

Esta captura muestra las estadísticas o sea un resumen de los empleados ya sea el total , el salario promedio, la cantidad por departamento o por cargo.

Metrics:

Total lines of code: 239
Total lines skipped (#nosec): 0

hardcoded_password_string: Possible hardcoded password: 'clave_secreta_123'
Test ID: B105
Severity: LOW
Confidence: MEDIUM
CWE: CWE-259
File: C:\Users\Paloma\Documents\Default Project\gestion_empleados\app.py
Line number: 7
More info: https://bandit.readthedocs.io/en/1.9.4/plugins/b105_hardcoded_password_string.html

```
6 app = Flask(__name__)
7 app.secret_key = "clave_secreta_123"
8
```

hashlib: Use of weak MD5 hash for security. Consider usedforsecurity=False
Test ID: B324
Severity: HIGH
Confidence: HIGH
CWE: CWE-327
File: C:\Users\Paloma\Documents\Default Project\gestion_empleados\app.py
Line number: 49
More info: https://bandit.readthedocs.io/en/1.9.4/plugins/b324_hashlib.html

```
48 def hash_password(password):
49     return hashlib.md5(password.encode()).hexdigest()
50
```

hardcoded_sql_expressions: Possible SQL injection vector through string-based query construction.
Test ID: B608
Severity: MEDIUM
Confidence: LOW
CWE: CWE-89
File: C:\Users\Paloma\Documents\Default Project\gestion_empleados\app.py
Line number: 54
More info: https://bandit.readthedocs.io/en/1.9.4/plugins/b608_hardcoded_sql_expressions.html

```
53 cursor = conn.cursor()
54 query = "SELECT * FROM usuarios WHERE username = '" + username + "' AND password = '" + hash_password(password) + "'"
55 cursor.execute(query)
```

hardcoded_sql_expressions: Possible SQL injection vector through string-based query construction.
Test ID: B608
Severity: MEDIUM

Esta captura muestra el bandit report, el cual detecto varios problemas los que estan por segun la gravedad es su color.

